Schema-Driven Document-Level Event Extraction with Coreferent Clusters: Relational Junction Scoring, Trigger-Cluster Coreference, and Balanced Positive-Negative Supervision

William Roe[1] (whr778@gmail.com), **Gemini**[2], and **Claude**[3] (noreply@anthropic.com)

[1] Project author and maintainer · [2] AI assistant (Google, Gemini) — diagnosis, architecture, experiments, and drafting · [3] AI assistant (Anthropic, Claude Opus 5) — design, implementation, and drafting

Paper 3 of the research programme stated in `RESEARCH_PROGRAM.md`. Drafted 2026-11-05, updated 2026-11-06; `eb22g` 5-epoch training at 95% (`bcfe500c`, Epochs 1–4 complete; Epoch 5 at step 78,419 / 82,930, ~2 hours of training + end-of-run calibration and blind test remaining).

Companion to `PAPER_0_FOUNDATION.md` (the GLiNER2 substrate and head-initialisation curve), `JOINT_IE_SCALING.md` (Paper 2 — the boundary-head data-scaling curve and constrained joint relation decoding), and `EKF_MHT_DESIGN.md` (Paper 1 — cross-document temporal state estimation, which consumes this model's document-level event records). Every experiment, positive, null, or negative, is recorded chronologically with its billed cost in `EXPERIMENT_CATALOG.md`.

Abstract

Schema-driven information extractors such as GLiNER2 take target labels as an inference-time prompt, avoiding closed ontologies, yet fail on document-level event extraction when multiple events of the same type occur in one document, when arguments lie hundreds of words from their triggers, or when an single real-world event is referred to by multiple coreferent trigger mentions across a passage. We trace this failure through four distinct architectural and optimization bottlenecks on a 4,096-token `mmBERT-base` boundary extractor and resolve each with a targeted, backwards-compatible mechanism:

1. **Instance individuation by trigger span (`event_records: true`).** Routing event supervision through the mention-structure path keys event instances by *event type*, pooling 64.2% of multi-instance document events into a single record and creating a 5× gap between relaxed (`0.5884`) and strict (`0.0991`) argument micro-F1 on `eb16-rebuild-tr`. Individuating each event instance by its trigger anchor span on the record head removes the 1-per-type pooling bottleneck without sacrificing open-schema prompting.
2. **Relational junction scoring (`record_link_mode: junction`).** Decomposing the record head's additive assignment logit `s_ijf = P_ij + R_fj` reveals that role fit `R_fj` (identical across every trigger in a group) accounts for 95% of within-row discrimination while pairwise trigger-argument compatibility `P_ij` sits below chance (per-column gold-vs-false trigger AUC `0.576`), allowing false triggers to steal 52–74% of arguments. Adding a role-gated bilinear junction term plus relative-order and log-distance geometry, trained with a contrastive per-column trigger softmax, lifts per-column junction AUC from `0.58` to `0.74–0.77` and unlocks English event argument extraction (`p5link` argument strict micro-F1 `+0.035`).
3. **Document-level coreferent trigger and argument clusters (`record_coreferent_ownership`, `TriggerLink`, `EntityCorefLink`).** A narrative article routinely mentions the same event multiple times ("Rebels *attacked* the base… The *assault* killed four"). We train every coreferent trigger mention to own its event's arguments, group coreferent and multi-occurrence argument mentions as occurrence alternatives of a single filler (`RecordFieldTarget.values`), and add a symmetric bilinear + MLP-distance `TriggerLink` scorer (pair AUC `0.802–0.843`) paired with validation-calibrated connected-component clustering (`coref_calibration.py`) and cluster-level evaluation metrics (`event_cluster` and `event_cluster_argument` strict micro-F1).
4. **Solving positive-gradient starvation under hard-negative and negative-query sampling (`eb22e` → `eb22f` → `eb22g`).** A paired role-candidate probe on `eb22e` (`3,568` gold role fillers) showed that under scheduled gold injection (`0.956`), `97.1%` of gold fillers sat in the candidate pool yet received a median assignment probability of `0.0155` because `_list_field_bce` averaged 1 positive with `K = 8` hard negatives in one denominator (equilibrium `P = 1/9 = 0.111`). Simultaneously, with gold injection off at inference, `61.7%` of gold fillers were missing from the pool or lost their trigger because `candidate_pair_loss` with `hard_negative_keep_all_when_absent: true` kept all 768 candidates on absent queries (`~85–90%` of queries under taxonomy negative sampling) in one global denominator, diluting positive pair gradients by 7,700× (`pair_loss` pinned at `0.005–0.011`) and leaving candidate logit levels unanchored under shift-invariant listwise losses. Splitting positive and negative reductions in both `_list_field_bce` and `candidate_pair_loss` (`pair_split_reduction: true`, `hard_negative_keep_all_when_absent: false`), withholding unresolved gold roles, and eliminating double event-schema dropout lifted 64-doc overfit `@0.5` strict micro-F1 on **CASIE** from `0.3619 -> 0.5210` (`+0.1591`, peaking at `0.5984`) on `event_trigger` and `0.2506 -> 0.4319` (`+0.1813`) on `event_argument`, and on **CMNEE** from `0.6375 -> 0.9394` (`+0.3019`) on `event_trigger` and `0.5470 -> 0.8005` (`+0.2535`) on

`event_argument`. On the full 263,949-document training mix (`eb22g`), Epoch 4 validation strict micro-F1 at swept threshold `@0.5` reaches `event_type` **`0.9524`**, `event_trigger` **`0.5656`** (precision `0.4594`, recall **`0.7358`**, fair **`0.5916`**), `event_argument` **`0.3046`** (precision `0.3202`, recall `0.2905`, relaxed **`0.5223`**, fair **`0.5080`** — nearly 5× `eb22d` Epoch 2's `0.0627`), `event` **`0.6317`** (relaxed **`0.6818`**), `event_cluster` **`0.5681`**, `event_cluster_argument` **`0.2655`** (4.6× `eb22d` Epoch 2's `0.0579`), `entity` **`0.3689`** (relaxed `0.4455`), `relation` **`0.2145`** (precision `0.6322`), `structure` **`0.2635`**, `classification` **`0.7166`**, `overall` **`0.4149`**, and `head_min` **`0.2145`**.

---

1. Introduction: Why Document-Level Events Fail in Open-Schema Extractors

Document-level event extraction asks a model to read a multi-paragraph document and emit a set of structured event records, each comprising an **event type**, one or more coreferent **trigger spans** anchoring the event in the text, and a mapping from **argument roles** to the entity or value spans that fill them. Unlike sentence-level benchmarks (ACE 2005; Walker et al., 2006) or closed-ontology taggers (OneIE; Lin et al., 2020), a generalist schema-driven extractor must satisfy four requirements simultaneously:

1. **Open schema at inference:** Event types and role inventories are supplied as prompt tokens at run time (`Schema.events({event_type: [roles]})`), not compiled into fixed classifier dimensions.
2. **Multi-instance individuation:** A news article or intelligence report often describes several distinct events of the same type (up to 16 `Attack` or `Military_Operation` instances in a single CMNEE or CASIE document).
3. **Long-range trigger-argument binding:** In document-level news (`cc_news_long`, CASIE, WikiEvents), the median trigger-to-argument distance is **446 words** (mean 1,333, 90th percentile 4,112 words; `EVENT_LINE.md`), so arguments routinely appear paragraphs away from the trigger they belong to.
4. **Cross-mention event and argument coreference:** A single real-world event is often evoked by multiple trigger mentions across a document (e.g., "*attacked*" in paragraph 1 and "*the assault*" in paragraph 4), each carrying partial role fillers that must be consolidated into one event cluster without fusing distinct events of the same type.

In earlier papers of this programme (`PAPER_0_FOUNDATION.md`, `JOINT_IE_SCALING.md`), moving from a 512-token DeBERTa-v3 span extractor to an 8,192-token `mmBERT-base` boundary extractor solved the 19-instance count ceiling and warmed the representations across 137,000 multi-task records, yet **`event_argument` strict micro-F1 remained stubbornly low (`0.069–0.118`) even when relaxed argument micro-F1 reached `0.506–0.588`**. This paper reports the series of diagnostic traces, negative results, and architectural/loss reforms (`eb16-rebuild-tr` → `eb17` → `eb18` → `eb21` → `eb22c/d/e/f/g`) that identified and solved the root causes of that gap.

---

2. Bottleneck 1: Instance Individuation by Trigger Span (`event_records: true`)

2.1 The Type-Pooling Defect in Mention-Path Events (Line 2 vs Line 3)

On the incumbent boundary model `whr778/gliner2-eb16-rebuild-tr` (18,786-document blind test, evaluated at its validation-calibrated threshold `0.2`), event extraction exhibited a nearly 6× split between strict and relaxed argument F1:

Metric (`eb16-rebuild-tr`, threshold `0.2`)	Strict Micro-F1	Relaxed Micro-F1	Fair (Ortmann) Micro-F1
`event_type`	`0.8650`	`0.8650`	—
`event_trigger`	`0.5984`	`0.6210`	`0.6180`
`event_argument`	**`0.0991`**	**`0.5884`**	**`0.5737`**

Strict evaluation keys each predicted argument by `(event_type, role, span, trigger)`, whereas relaxed evaluation drops the trigger link and checks surface overlap under the event type (`EVENT_ARGUMENT_DIAGNOSIS.md`). Tracing the training and decode path revealed why: while `enable_records: true` was set in the boundary head, `event_records: true` was never enabled in the base training configs. Consequently, events were supervised through the mention-structure path, which **individuates instances by `event_type` (at most 1 instance per type per window)** and pools all triggers and arguments of the same event type into a single bag.

Across the multi-event corpora in our mixture, **64.2% of gold event instances share their event type with at least one other instance in the same document** (39.9% even share the same trigger surface string in a different sentence). Pooling them into one record per type made nearly two-thirds of gold event instances structurally inexpressible.

2.2 Anchor-Seeded Record Instances (`event_records: true`)

Enabling `event_records: true` routes event types through the boundary record head (`gliner2/models/boundary/records.py`) in `natural` (`per_query`) mode: - Role index `0` is always the **trigger** query, and roles `1..F` are the event's **argument** queries. - Each trigger candidate span `t_i` proposed for the event type seeds a distinct **event instance hypothesis** (up to `record_instance_queries = 32` per event type per window, comfortably above the worst observed document density of 16 same-type events). - Arguments are bound to specific trigger instances `t_i` via an instance-by-candidate assignment tensor `s_ijf` over trigger instance `i` , candidate span `a_j` , and role field `f` .

---

3. Bottleneck 2: Relational Junction Scoring (`record_link_mode: junction`)

3.1 Why Additive Assignment Cannot Bind Arguments to Triggers

Simply turning on `event_records: true` (`eb17` , `eb18`) was not enough to bind English event arguments. In `eb18` , the record head scored trigger instance `t_i` against candidate span `a_j` for role field `r_f` additively (`records.py` `_assign_logits`):

```
s_ijf (additive) = P_ij + R_fj
                 = inst_proj(t_i) · cand_proj(a_j)   [pairwise P_ij]
                 + field_proj(r_f) · cand_proj(a_j)  [role fit R_fj, identical for all i]
```

We decomposed `s_ijf` on 16 real training batches (`456` gold arguments, `3,648` false-trigger comparisons; `JUNCTION_LAYER_SPEC.md` §1):

Diagnostic Metric	**`eb18`**	**`p3arg-hard`**
Variance across triggers `i` vs across candidates `j`	`18.7` vs `44.1`	`4.3` vs `12.8`
Per-column Junction AUC (gold trigger vs false trigger on same gold argument `a_j`)	**`0.631`**	**`0.576`**
Per-row Argument AUC (`R_fj` only / `P_ij` only / `P_ij + R_fj`)	`0.947` / **`0.405`** / `0.948`	`0.953` / **`0.371`** / `0.954`

Because `R_fj` does not depend on trigger `i` , **the additive score is 95% role fit with no relational join**. A candidate span that looks like a plausible `Victim` or `Attacker` receives virtually the same high score on *every* trigger row in the document, so false or sibling triggers steal 52–74% of arguments, and pushing false trigger rows down in training depresses the shared role term `R_fj` and destroys recall.

3.2 Gated Bilinear + Geometry Junction Layer and Contrastive Column Loss

In relational terms, `Trigger` (`t_i`) and `Argument` (`a_j`) are many-to-many entities that require an explicit join table keyed by `(i, j, f)` . Under `record_link_mode: junction` , we augment the assignment logit with a **role-gated bilinear link** and a **relative geometry** term:

```
s_ijf (junction) = P_ij + R_fj
                 + <(U t_i) ⊙ (V a_j) ⊙ σ(G q_f), 1> / sqrt(d)   [gated bilinear link]
                 + w_f · e_geom(i, j)                            [order + log distance]
```

where `e_geom(i, j)` encodes relative span order and log-bucketed token distance between trigger span `t_i` and argument span `a_j` , and `V, w_f` are zero-initialized so warm starts are bit-identical at step 0.

Crucially, rather than training false trigger rows with absolute negative BCE targets (which depresses `R_fj`), we supervise the junction **contrastively down each gold argument's column**: for every gold argument column `j` of role `f` , a listwise cross-entropy loss (`record_link_column_weight: 0.3`) maximizes the log-probability mass assigned to the gold trigger instance(s) `G_j` against all competing trigger instances in that column.

In a controlled A/B (`p5link` , 2026-10-04), enabling the junction layer and column loss lifted per-column junction AUC from `0.58` to **`0.74–0.77`** and improved `event_argument` strict micro-F1 by **`+0.035`** (with `eb22f` and `eb22g` maintaining cumulative junction AUC of **`0.728–0.760`** over 200,000+ natural groups).

---

4. Bottleneck 3: Document-Level Coreferent Event and Argument Clusters

4.1 Multi-Mention Event Ownership (`record_coreferent_ownership: true`)

Real-world news articles frequently refer to the same event through multiple coreferent triggers across a document (`cc_news_events_sonnet55_v2` contains 9,528 positive coreferent trigger pairs across 3,760 multi-mention events). Previously, when an

event record listed multiple coreferent triggers `triggers = [t_1, t_2, ...]`, `compute_group_loss` seeded a gold instance only on `t_1` (`cols[0]`): `t_2` was protected from negative supervision but received **zero argument training**, and at decode time both `t_1` and `t_2` could fire as separate duplicate events that split the event's arguments between them (`COREFERENT_OWNERSHIP_SPEC.md`).

Under `record_coreferent_ownership: true`, **every coreferent trigger mention `t_m` in cluster `G` seeds a gold instance row and owns the event's argument targets**, with the record's field loss normalized by `1 / |G|` so multi-mention events do not overweight the gradient.

4.2 Learned Trigger Coreference (`TriggerLink`) and Validation-Calibrated Clustering

Because `record_coreferent_ownership: true` trains every trigger mention of an event to emit its arguments, decode must consolidate mentions of the *same* event into a single cluster without merging distinct same-type events.

A first heuristic — merging same-type decoded instances that shared a high-scoring argument (`3b-i`) — **traced negative**: on 15 multi-trigger validation documents, **0 of 6 merged events matched a gold cluster**, because argument-score cosine is dominated by the shared role term `R_fj` (pairwise AUC **`0.468`**, below chance; `COREFERENT_LINK_SPEC.md` §1). By contrast, the cosine of `inst_proj` trigger states already carried an untrained coreference AUC of **`0.709`**.

We therefore added `TriggerLink` (`record_coref_link: true`), which scores trigger instance pairs `(t_i, t_j)` within an event type via a symmetric bilinear projection plus a 64-width MLP over pair geometry (`coref_link_distance: mlp`, `coref_link_distance_dim: 64`, taking `log(1 + Δ_tok)`, same-sentence indicator, and surface-match indicator):

```
s_ij (coref) = <W t_i, W t_j> / sqrt(d) + MLP_geom(g(i, j))
```

Why an MLP for distance? In the `eb22c` score-level autopsy (`EB22C_SCORE_LEVEL_AUTOPSY.md` §10), a linear scalar distance weight accounted for 66–68% of the coreference logit; replacing it with a zero-initialized 64-unit MLP was the **only** `eb22c` architectural arm that cleared the three-control noise floor on both validation corpora (catalog 2026-10-23). Across `eb22f` and `eb22g`, `TriggerLink` achieves **pair AUC `0.802–0.843`** over `19.8M` mention pairs.

At the end of training, `gliner2/training/coref_calibration.py` sweeps the merge threshold `τ_coref` on validation (`cc_news_events_sonnet55_v2`), gated by: 1. At least 20 merges performed, 2. Pairwise merge precision ≥ 0.80, 3. Distinct-event fusion rate < 0.05, and 4. Non-decreasing cluster F1 on both the calibration set and single-trigger controls (`cmnee`, `maven`).

If no threshold clears all four gates, `record_merge_coreferent` remains `"off"` in `config.json` rather than shipping a destructive merge.

4.3 Argument Coreference and Multi-Occurrence Target Compilation

Document-level benchmarks (WikiEvents, RAMS, ACE, and our LLM-annotated news corpora) score an extracted argument as correct if it matches *any* coreferential mention of the gold entity, or if the canonical entity surface appears multiple times in the document. Our audit (`2026-11-04`, commit `0607b29`) uncovered a critical target-compilation bug in how both coreferent argument mentions (`expand_argument_coreferents` in `gliner2/processing/argument_coref.py`) and multi-occurrence string-gold arguments (`_build_outputs` in `gliner2/processor.py`) were handed to the record head:

- **Before (`eb22d` – `eb22f`):** When `expand_argument_coreferents` appended a coreferent mention `"he"` to `"John Smith"`, or when a string-gold argument `"malware"` appeared at 4 token spans in a CASIE document, `_build_record_field_target` (`boundary_preprocessing.py`) compiled each occurrence as a **separate required filler** in `RecordFieldTarget.values` (`len(ft.values) = K`). During training, `_list_field_bce` then forced the model to push *all `K` columns simultaneously toward `1.0`* for a single role, and treated any occurrence that fell outside the candidate pool as an unresolved missing role!
- **After (`0607b29` / `eb22g`):** `expand_argument_coreferents` tags coreferent expansions with `_coref_group`, and `_build_outputs` wraps each listed argument's document matches as one occurrence group, so `_build_record_field_target` compiles **one** value entry whose tuple holds all acceptable span alternatives: `ft.values = (((span_1), (span_2), ...),)` (`len(ft.values) = 1`). Resolving *any* coreferent or repeated mention in the candidate pool satisfies the role filler.

---

5. Bottleneck 4: Positive-Gradient Starvation in Role Assignment and Span Proposal

5.1 The `eb22e` Paired Role-Candidate Probe: Separating Assignment from Proposal

After zero-initializing the post-`eb21` span residuals (`span_residual_zero_init: true`, `enable_rotary_endpoints: false` in `eb22e`), Epoch 1 validation `event_argument` strict micro-F1 sat at `0.0755` (at swept threshold `0.1`), and blind-test evaluation on WikiEvents at flat threshold `0.5` scored `0.0000` on every event head (`EXPERIMENT_CATALOG.md` 2026-11-02).

To separate **role-assignment failure** from **span-proposal failure** without confounding from random schema dropout, we ran a **paired-collate probe** (`probe_role_candidates.py` , `2026-11-03`) on `eb22e:checkpoint-epoch-1` across `3,568` gold role fillers (`20` val docs each of `casie` , `cmnee` , `duee` + all `19` `wikievents.dev`), collating each batch **once** and running two forwards — first at Epoch 1's scheduled gold injection (`0.9559`), then with gold injection off (`0.0` , the inference regime):

Regime (`n = 3,568` identical gold fillers)	Candidate in Pool	Not in Pool	Trigger Not Proposed	Median Assignment `P`	Mean Assignment `P`	Share `P ≥ 0.1`	Share `P ≥ 0.5`
Scheduled injection (`0.9559`)	**`97.1%`** (`3,466`)	`2.3%`	`0.6%`	**`0.0155`**	`0.0674`	`18.0%`	**`2.1%`**
Inference (`0.0` injection)	**`38.3%`** (`1,366`)	**`28.8%`**	**`32.9%`**	**`0.0280`**	`0.0994`	`25.2%`	**`4.4%`**

Per-corpus candidate reachability at `0.0` injection exposed a sharp split: - **Chinese sentence/short-doc corpora:** `duee` **`100.0%`** candidate (`median P = 0.0684`), `cmnee` **`96.4%`** candidate (`2.4%` not in pool, `1.2%` trigger missing, `median P = 0.0452`). - **English multi-paragraph corpora:** `casie` **`34.8%`** candidate (`29.5%` not in pool, `35.7%` trigger missing, `median P = 0.0048`), `wikievents` **`30.0%`** candidate (`32.9%` not in pool, `37.1%` trigger missing, `median P = 0.0352`).

This single table proved that **two independent starvations** were operating at the same time: 1. **Even when `97.1%` of gold fillers were present in the candidate pool, their median assignment probability was `0.0155`** (only `2.1%` reached `0.5`). 2. **When gold injection was turned off at inference, `61.7%` of gold fillers were not candidates at all** (`28.8%` span not in pool, `32.9%` trigger not proposed).

5.2 Solving Role-Assignment Starvation (`overfit-events-eb22e` → `eb22f`)

Tracing `_list_field_bce` (`gliner2/models/boundary/records.py`) and `_format_field` (`gliner2/inference/engine.py`) identified four defects depressing role assignment scores:

1. **Combined `(1 + K)` denominator in `_list_field_bce` :** With `record_role_hard_negatives: 8` , `_list_field_bce` concatenated `G` gold columns (`target = 1`) and `K = 8` hardest negative columns (`target = 0`) and took a single `.mean()` . For a 1-filler role (`G = 1, K = 8`), the positive gradient weight was `1/9` while the negative weight was `8/9` , placing the zero-gradient equilibrium at `P = 1 / (1 + K) = 0.111` whenever hard negatives shared features with the gold filler. **Fix:** Split the reduction into `pos_loss.mean() + neg_loss.mean()` , giving the positive set and hard-negative set equal total weight regardless of `G` and `K` .
2. **Training missed gold roles as empty (`target = 0`):** When a gold role had `len(ft.values) > 0` but none of its spans entered the candidate pool (`gold_cols = []`), `_list_field_bce` trained the hardest candidate columns toward `0.0` , teaching the model to suppress valid role fillers whenever the proposer missed the exact gold boundaries, and masking out partially overlapping spans on partial matches. **Fix:** Withhold unresolved non-empty roles (`0.0` loss) in both sparse (`compute_group_loss`) and dense (`compute_dense_batch_loss` / `build_dense_record_matching_cost`) paths, and mask candidates that overlap unresolved gold spans on partial matches (`_unresolved_overlap_cols`).
3. **Anchor self-row in field loss (`record_drop_anchor_field_loss: true`):** In `natural` mode, `field_idx = 0` is the trigger anchor that seeded the instance, yet its row was included in `field_loss_sum / len(spec.fields)` . Dropping the anchor self-row focuses `record_field_loss` exclusively on argument roles.
4. **Double-gating at format time (`engine.py`):** `decode_group` already gated role fillers by `assign_prob = sigmoid(assign_logit / T) >= threshold` , and `engine.py` then overwrote the filler's confidence with `min(assign_prob, span_score)` (where `span_score` was the uncalibrated raw pair score), causing `_format_field` to discard valid arguments whose assignment probability cleared the threshold. **Fix:** Store the assignment probability as the field confidence (`span_score` preserved as `pair_confidence` diagnostic).

Experimental validation (`overfit-events-eb22e` , 1x A10, 64 docs × 400 steps on `eb22e:checkpoint-epoch-1`):

Corpus (64 docs, Step 400, injection off at eval)	Arm	Median Gold Role P	Share P ≥ 0.5	event_argument Strict Micro-F1 @0.5	event_argument Strict Micro-F1 @0.3	event_argument Strict Micro-F1 @0.1
casie_off	Control (legacy role loss)	0.0316	13.5%	0.1389	0.1779	0.2437
casie_off	**Treatment (eb22f role loss + gate)**	**0.5545**	**53.4%**	**0.1938** (+0.0549)	**0.2407** (+0.0628)	**0.2567** (+0.0130)
cmnee_ner_off	Control (legacy role loss)	0.5078	50.4%	0.3119	0.4000	0.5238
cmnee_ner_off	**Treatment (eb22f role loss + gate)**	**0.9196**	**82.0%**	**0.5778** (+0.2659)	**0.5882** (+0.1882)	**0.5872** (+0.0634)

Even at **Step 0** (frozen checkpoint, isolating the `_format_field` confidence fix alone), `cmnee_ner_off` argument strict micro-F1 `@0.5` jumped from **`0.105 -> 0.190`** (+0.085). By Step 400, median gold role assignment probability rose **`0.0316 -> 0.5545`** on CASIE and **`0.5078 -> 0.9196`** on CMNEE (`+0.266` argument F1 `@0.5`).

However, when we trained a full Epoch 1 of the 263,949-document mix with these fixes (`eb22f-epoch`, 4x H100 SXM5, `step 16,585`), validation strict micro-F1 at `@0.1` improved substantially on **`entity (0.4262 -> 0.4679, +0.0417)`** and **`relation (0.1326 -> 0.1665, +0.0339)`**, while **`event_argument` remained at `0.0767` (vs `eb22e`'s `0.0755`)** and `event_trigger` remained at `0.1619` (vs `0.1608`). That confirmed the second half of the paired probe's prediction: during Epoch 1, gold injection stays at `1.0 -> 0.96`, and without fixing **why the proposer fails to propose 61.7% of gold spans when injection is off**, role-loss improvements cannot surface at inference.

5.3 Solving Candidate Proposal Starvation (`overfit-events-eb22g` → `eb22g`)

Why was the boundary proposer failing to propose English event triggers and arguments — and why had `eb22c` exhibited score-level swings where `pointwise pair_loss` sat at `~0.005` against `listwise ~3.5` (`EB22C_SCORE_LEVEL_AUTOPSY.md` §4)?

Tracing `candidate_pair_loss` (`gliner2/models/boundary/losses.py`) and `processor.py` revealed the exact mechanism:

1. **Absent-query negative dominance in `candidate_pair_loss`:** Under taxonomy negative sampling (`negative_labels_per_dim: {entities: 20, events: 20}` + `gold_full_roles: true`), 85–90% of queries in a training batch have **zero positive spans** (`pos_count == 0`). In `eb22c`–`eb22f`, `hard_negative_keep_all_when_absent: true` kept **all 768 candidates** as negatives on every absent query, and `candidate_pair_loss` divided the sum of all positive and negative BCE losses by `total_mask.sum()`. - **Traced on 1 positive span at logit `-5.0` + 10 absent queries × 768 candidates at logit `-3.0`:** `total_mask.sum()` is `1 + 20 + 7,680 = 7,701`. The positive span's BCE loss (`5.0067`) is divided by `7,701`, shrinking its gradient on the positive logit from `-0.9933` (`= P − 1`) to **`-0.000129`** (7,700× smaller!) and shrinking `pair_loss` to **`0.0492`**. - Meanwhile, the listwise losses (`proposal_loss` and `rerank_listwise_loss`) are softmaxes over candidates within positive queries only — which are **shift-invariant** (`softmax(s + c) = softmax(s)`). With the only level-anchoring positive term crushed by 7,700× and absent queries pushing all candidates down, the overall logit level drifted below the `−0.85` (`threshold = 0.3`) decode gate! - **Fix (`eb22g`):** Added `pair_split_reduction: true` (`split_positive_negative=True` in `candidate_pair_loss`), which averages positive BCE over `pos_mask.sum()` and negative BCE over `neg_mask.sum()` (`0.5 * (pos_mean + neg_mean)` when both exist), and set `hard_negative_keep_all_when_absent: false` so absent queries sample at most `minimum_hard_negatives: 16` hardest false positives instead of all 768 candidates. On the same traced batch, `pair_loss` moves **`0.0492 -> 5.0553`** and the positive logit gradient restores to **`-0.993307`**.
2. **Double event-schema dropout (`remove_events_prob: 0.0`):** Even after `NegativePoolSampler` constructed the training menu, `SchemaDropout` in `_process_events` randomly dropped `20%` of event types (`remove_events_prob: 0.2`) — discarding a fifth of gold event supervision on every step, and even firing inside the inference dict-schema branch if `is_training` remained set! We removed `remove_events_prob` from the inference branch and set `remove_events_prob: 0.0` in `eb22g.yaml`, increasing supervised gold role fillers per step by ~25%.
3. **Evaluation hygiene (`train.py`, `eval_metrics.py`):** Fixed `_gold_event_trigger_set`, `_gold_event_argument_set`, and `_gold_oneie_sets` to unwrap `{text, start, end}` offset dicts (`casie_off` perfect-prediction F1 `0.0 -> 1.0`), and fixed standalone `evaluate_config` to exclude `data.train_only` corpora and run `check_and_clean`.

6. Experimental Results

6.1 Controlled 64-Document Overfit A/B: `overfit-events-eb22g` (`eb22f` vs `eb22g`)

Before committing to a 5-epoch training run, we ran a controlled 4-arm experiment (`overfit-events-eb22g` , `1x A100 40 GB SXM4` , instance `37b84a7e` , commit `a0d8e11`) starting from the exact same `whr778/gliner2-eb22f-epoch:checkpoint-epoch-1` weights across 64 training documents of **`casie_off`** (English multi-paragraph cybersecurity news: 609 gold triggers, 1,518 gold arguments) and **`cmnee_ner_off`** (Chinese military news: 122 gold triggers, 399 gold arguments) for 400 steps (`batch_size = 4` , `grad_accum = 1`). Both arms included the occurrence/coref target grouping (`0607b29`); **Control** used the `eb22f` settings (`pair_split_reduction=False` , `hard_negative_keep_all_when_absent=True` , `remove_events_prob=0.2`) and **Treatment** used the `eb22g` settings (`pair_split_reduction=True` , `hard_negative_keep_all_when_absent=False` , `remove_events_prob=0.0`). All evaluations ran with gold injection **off** (`is_training=False`):

Table 1: `casie_off` (64 records: 609 triggers, 1,518 arguments — Strict Micro-F1)

Metric	Prior `eb22e` Treatment (Step 400)	Step 0 (`eb22f` ckpt)	Step 400 Control (`eb22f`)	Step 400 Treatment (`eb22g`)	Delta (`eb22g` − `eb22f`)
`event_type` strict micro-F1 `@0.5`	`0.8618`	`0.0000`	`0.9384`	**`1.0000`**	**`+0.0616`**
`event_type` strict micro-F1 `@0.3`	`0.9163`	`0.0000`	`0.9533`	**`1.0000`**	**`+0.0467`**
`event_type` strict micro-F1 `@0.1`	`0.9609`	`0.0177`	`0.9581`	**`1.0000`**	**`+0.0419`**
`event_trigger` strict micro-F1 `@0.5`	`0.3889`	`0.0000`	`0.3619`	**`0.5210`** *(step 350: **`0.5984`**)*	**`+0.1591`**
`event_trigger` strict micro-F1 `@0.3`	`0.3824`	`0.0000`	`0.3877`	**`0.4554`** *(step 350: **`0.5506`**)*	**`+0.0677`**
`event_trigger` strict micro-F1 `@0.1`	`0.3712`	`0.0037`	`0.3982`	`0.3731` *(step 350: **`0.4741`**)*	`−0.0251`
`event_argument` strict micro-F1 `@0.5`	`0.1938`	`0.0000`	`0.2506`	**`0.4319`**	**`+0.1813`**
`event_argument` strict micro-F1 `@0.3`	`0.2407`	`0.0000`	`0.2871`	**`0.4108`**	**`+0.1237`**
`event_argument` strict micro-F1 `@0.1`	`0.2567`	`0.0025`	`0.2857`	**`0.3303`** *(step 350: **`0.4110`**)*	**`+0.0446`**
`pair_loss` (step 50 → step 400)	—	`0.0000`	`0.0109 -> 0.0113` *(flat)*	**`1.6638 -> 0.4118`**	actively trains
`record_field_loss` (step 400)	`0.3316`	`0.0000`	`0.3093`	**`0.2793`**	**`−0.0300`**
`gold_role_n` (steps 350–400)	`3,967`	—	`4,149`	**`5,119`**	**`+970 (+23.4%)`**
`gold_role_p_median` (step 400)	`0.5545`	—	`0.6316`	**`0.7058`**	**`+0.0742`**

Table 2: `cmnee_ner_off` (64 records: 122 triggers, 399 arguments — Strict Micro-F1)

Metric	Prior `eb22e` Treatment (Step 400)	Step 0 (`eb22f` ckpt)	Step 400 Control (`eb22f`)	Step 400 Treatment (`eb22g`)	Delta (`eb22g − eb22f`)
`event_type` strict micro-F1 `@0.5`	`0.8584`	`0.1798`	`0.7538`	**`0.9811`**	**`+0.2273`**
`event_type` strict micro-F1 `@0.3`	`0.8780`	`0.2198`	`0.8000`	**`0.9938`**	**`+0.1938`**
`event_type` strict micro-F1 `@0.1`	`0.9032`	`0.4118`	`0.8175`	**`0.9938`**	**`+0.1763`**
`event_trigger` strict micro-F1 `@0.5`	`0.6512`	`0.1308`	`0.6375`	**`0.9394`**	**`+0.3019`**
`event_trigger` strict micro-F1 `@0.3`	`0.6909`	`0.1622`	`0.6905`	**`0.9100`**	**`+0.2195`**
`event_trigger` strict micro-F1 `@0.1`	`0.7200`	`0.2927`	`0.7119`	**`0.8276`** *(step 250: **`0.8829`**)*	**`+0.1157`**
`event_argument` strict micro-F1 `@0.5`	`0.5778`	`0.0964`	`0.5470`	**`0.8005`**	**`+0.2535`**
`event_argument` strict micro-F1 `@0.3`	`0.5882`	`0.1137`	`0.5968`	**`0.7543`** *(step 300: **`0.8099`**)*	**`+0.1575`**
`event_argument` strict micro-F1 `@0.1`	`0.5859`	`0.1367`	`0.5859`	**`0.6780`** *(step 300: **`0.7786`**)*	**`+0.0921`**
`pair_loss` (step 50 → step 400)	—	`0.0000`	`0.0052 -> 0.0060` *(flat)*	**`1.1424 -> 0.2650`**	actively trains
`record_field_loss` (step 400)	`0.5625`	`0.0000`	`0.5092`	**`0.3545`**	**`−0.1547`**
`gold_role_n` (steps 350–400)	`1,100`	—	`952`	**`1,260`**	**`+308 (+32.4%)`**
`gold_role_p_median` (step 400)	`0.9196`	—	`0.9763`	**`0.9797`**	**`+0.0034`**

Three findings stand out from Tables 1 and 2: 1. **Occurrence/coreferent target grouping alone lifted `casie_off` `@0.5` argument F1 from `0.1938 -> 0.2506` (`+0.0568`)** between `overfit-events-eb22e` 's treatment and `overfit-events-eb22g` 's control. 2. **Restoring positive `pair_loss` broke the `~0.25` CASIE proposal ceiling**, lifting `casie_off` `@0.5` trigger strict micro-F1 to **`0.5210`** (`0.5984` at step 350) and argument strict micro-F1 to **`0.4319`** (`+0.1813` over control, `+0.2381` over `overfit-events-eb22e`). 3. **Threshold inversion disappeared:** Because positive pair logits are now anchored above `0.0` , `@0.5` is the strongest operating point on both CASIE (`0.4319` vs `0.3303` at `@0.1`) and CMNEE (`0.8005` vs `0.6780` at `@0.1`).

6.2 Full-Scale Training Progression: `eb22d` vs `eb22e` vs `eb22f` vs `eb22g` (Epochs 1–4, Live)

We trained `eb22g` from scratch (`from_encoder` on `jhu-clsp/mmBERT-base` , seed 42, `263,949` cleaned training records → `265,369` 4,096-token chunks, `19,286` validation records, `20,602` blind-test records, `82,930` steps across 5 epochs) on `1x A100 40 GB SXM4` (`bcfe500c`). At step 20, `boundary_head.pair_scorer` registered a gradient norm of `34.8` (4th largest in the model), compared to near-zero in `eb22d` – `eb22f` .

Table 3 compares the 1,500-document validation draw across **Epochs 1–4 (`steps 16,586` to `66,344`)** of `eb22g` against the earlier `eb22` runs (single-run noise floor **±0.02**, relations **±0.041**):

Table 3: Full-Mix Validation Progression Across the `eb22` Series

Metric (Validation, Strict Micro-F1 unless noted)	eb22d Ep 1 (@0.1)	eb22e Ep 1 (@0.1)	eb22f Ep 1 (@0.1)	eb22d Ep 2 (@0.3 full val)	eb22g Ep 1 (@0.3)	eb22g Ep 2 (@0.3)	eb22g Ep 3 (@0.5)	eb22g Ep 4 (@0.5)	Delta (eb22g Ep 4 − eb22d Ep 2)
Selected Swept Threshold	0.1	0.1	0.1	0.3	**0.3**	**0.3**	**0.5**	**0.5**	Standard 0.5
eval_loss	—	1.0012	0.9923	—	1.6277	1.4125	1.2381	**1.2268**	Monotonic descent
event_argument strict micro-F1	0.1123	0.0755	0.0767	0.0627	0.0974	0.1639	0.2181	**0.3046**	**+0.2419** *(4.86× eb22d Ep 2)*
event_argument strict Precision / Recall	—	0.237 / 0.045	0.142 / 0.053	0.296 / 0.035	0.063 / 0.221	0.115 / 0.284	0.167 / 0.316	**0.320 / 0.291**	Balanced P= 32.0%, R= 29.1%
event_argument relaxed micro-F1	—	0.1021	0.0985	—	0.2620	0.3992	0.4801	**0.5223**	**+0.4238 vs eb22f Ep 1**
event_argument fair (Ortmann) micro-F1	—	0.0989	0.1026	—	0.2191	0.3632	0.4544	**0.5080**	**+0.4054 vs eb22f Ep 1**
event_type strict micro-F1	0.6478	0.5988	0.3848	—	0.9681	0.9762	0.9654	**0.9524**	**+0.5676 vs eb22f Ep 1**
event_trigger strict micro-F1	0.1608	0.1608	0.1619	0.3645 *(R= **0.316**)*	0.1610	0.2522	0.4113	**0.5656** *(P= **0.459**, R= **0.736**, fair= **0.592**)*	**+0.2011** *(+55% relative)*
event (combined) strict micro-F1	—	—	0.2109	—	0.2543	0.3505	0.4945	**0.6317** *(rel **0.6818**)*	**+0.4208 vs eb22f Ep 1**
event_cluster strict micro-F1	—	—	0.1729	0.3660	0.1694	0.2584	0.4149	**0.5681** *(P= **0.448**, R= **0.775**)*	**+0.2021 vs eb22d Ep 2**
event_cluster_argument strict micro-F1	—	—	0.0635	0.0579	0.0821	0.1427	0.1878	**0.2655** *(P= **0.273**, R= **0.259**)*	**+0.2076** *(4.59× eb22d Ep 2)*
classification strict micro-F1	0.4921	0.4802	0.4645	—	0.4574	0.5442	0.6794	**0.7166**	**+0.2521 vs eb22f Ep 1**
relation strict micro-F1	0.1340	0.1326	0.1665	—	0.0448	0.0562	0.2058	**0.2145** *(P= **0.632**, rel **0.2681**)*	**+0.0480 vs eb22f Ep 1**
structure strict micro-F1	0.1691	0.2231	0.1501	—	0.0648	0.0847	0.1957	**0.2635** *(rel **0.2893**)*	**+0.1134 vs eb22f Ep 1**
entity strict micro-F1	0.3617 *(@0.1)*	0.4262 *(@0.1)*	0.4679 *(@0.1)*	—	0.2867 *(@0.3)*	0.2445 *(@0.3)*	0.3208 *(@0.5)*	**0.3689** *(@0.5, rel **0.4455**, fair **0.4234**)*	**+0.0481 Ep 3 → Ep 4**

Metric (Validation, Strict Micro-F1 unless noted)	eb22d Ep 1 (@0.1)	eb22e Ep 1 (@0.1)	eb22f Ep 1 (@0.1)	eb22d Ep 2 (@0.3 full val)	eb22g Ep 1 (@0.3)	eb22g Ep 2 (@0.3)	eb22g Ep 3 (@0.5)	eb22g Ep 4 (@0.5)	Delta (eb22g Ep 4 − eb22d Ep 2)
overall strict micro-F1	—	—	0.4017 *(@0.1)*	—	0.2748	0.2627	0.3576	**0.4149** *(rel **0.4811**)*	**+0.0573** Ep 3 → Ep 4
head_min (eval_overall_strict_head_min_f1)	—	0.0755	0.0767	—	0.0448	0.0562	0.1957	**0.2145**	**+0.1378** over eb22f *(2.80×)*

(Epoch 5 [step 82,930, ~2 hours away], post-training threshold_sweep.json + per-head thresholds, coref_calibration.json, and the 20,602-record blind test test_metrics.json will be appended here upon completion of bcfe500c.)

---

7. Downstream Connection: Providing the Event Substrate for EKF/MHT

In RESEARCH_PROGRAM.md and EKF_MHT_DESIGN.md, the cross-document temporal tracker (Paper 1) requires an upstream extractor that converts each news article into structured event observations (event_type, trigger_cluster, role -> value) before the Viterbi association and Kalman filter stages run. Earlier EKF front-end runs (EVENT_LINE.md §4a) were blocked because: 1. Mention-path event extraction pooled all events of the same type in an article into one record, blending casualties or locations from separate incidents; 2. Without the junction layer, false triggers stole arguments from true triggers; and 3. Without TriggerLink coreference and positive-gradient proposal supervision, multi-mention events either fragmented into duplicate observations or failed to propose spans on long English news articles.

With eb22g uniting trigger-anchored record instances, relational junction scoring, coreferent trigger/argument clustering, and balanced positive-negative proposal and role supervision, the event extraction front-end now meets the structural contract required to resume the EKF/MHT multi-event tracking experiments once Epoch 5 completes.

---

8. Reproducibility & Code Map

Mechanism	Primary Implementation	Config Flag (`base/eb22g.yaml`)
Trigger-seeded event records	`gliner2/models/boundary/records.py` (`forward_group` , `compute_group_loss` , `decode_group`)	`enable_records: true` , `event_records: true` , `candidate_pool: per_query`
Relational junction layer & column loss	`gliner2/models/boundary/records.py` (`_assign_logits` , `_column_loss`)	`record_link_mode: junction` , `record_link_column_weight: 0.3`
Multi-mention trigger ownership	`gliner2/models/boundary/records.py` (`compute_group_loss`)	`record_coreferent_ownership: true`
Learned trigger coreference & MLP distance	`gliner2/models/boundary/records.py` (`TriggerLink`)	`record_coref_link: true` , `coref_link_distance: mlp` , `coref_link_distance_dim: 64`
Validation-calibrated cluster merge	`gliner2/training/coref_calibration.py` , `tools/train/train.py`	`eval.coref_calibration` , `eval.coref_control`
Argument coreference & occurrence grouping	`gliner2/processing/argument_coref.py` , `gliner2/processor.py` , `gliner2/processing/boundary_preprocessing.py`	`training.expand_argument_coref: true`
Split positive/hard-negative role BCE & withhold unresolved	`gliner2/models/boundary/records.py` (`_list_field_bce` , `_unresolved_overlap_cols`), `matching.py`	`record_role_hard_negatives: 8` , `record_drop_anchor_field_loss: true`
Split positive/negative pair loss & absent-query cap	`gliner2/models/boundary/losses.py` (`candidate_pair_loss`), `model.py`	`pair_split_reduction: true` , `hard_negative_keep_all_when_absent: false`
Assignment-probability decode gate	`gliner2/inference/engine.py` (`_format_record_extractions`)	default in record formatting
Cluster & offset-gold evaluation metrics	`gliner2/training/eval_metrics.py` (`_gold_event_trigger_set` , `_gold_event_argument_set` , `event_cluster*`)	default in `compute_metrics`

9. References

Methods and Models

- Zaratiana, U., Pasternak, G., Boyd, O., Hurn-Maloney, G., Lewis, A. (2025). *GLiNER2: Schema-Driven Multi-Task Learning for Structured Information Extraction.* EMNLP 2025 System Demonstrations, pp. 130–140. ACL. https://aclanthology.org/2025.emnlp-demos.10/
- Zaratiana, U., Tomeh, N., Holat, P., Charnois, T. (2024). *GLiNER: Generalist Model for Named Entity Recognition using Bidirectional Transformer.* NAACL. https://aclanthology.org/2024.naacl-long.300/
- Marone, M., Weller, O., Fleshman, W., Yang, E., Lawrie, D., Van Durme, B. (2025). *mmBERT: A Modern Multilingual Encoder with Annealed Language Learning.* arXiv:2509.06888. https://arxiv.org/abs/2509.06888
- Warner, B., et al. (2024). *Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference* (ModernBERT). arXiv:2412.13663. https://arxiv.org/abs/2412.13663
- Lin, Y., Ji, H., Huang, F., Wu, L. (2020). *A Joint Neural Model for Information Extraction with Global Features* (OneIE). ACL. https://aclanthology.org/2020.acl-main.713/
- Ortmann, K. (2022). *Fine-Grained Error Analysis and Fair Evaluation of Labeled Spans.* LREC. https://aclanthology.org/2022.lrec-1.150/
- Dozat, T., Manning, C. D. (2017). *Deep Biaffine Attention for Neural Dependency Parsing.* ICLR. https://openreview.net/forum?id=Hk95PK9le
- Lee, K., He, L., Lewis, M., Zettlemoyer, L. (2017). *End-to-end Neural Coreference Resolution.* EMNLP. https://aclanthology.org/D17-1018/
- Kuhn, H. W. (1955). *The Hungarian Method for the Assignment Problem.* Naval Research Logistics Quarterly, 2(1–2), 83–97.

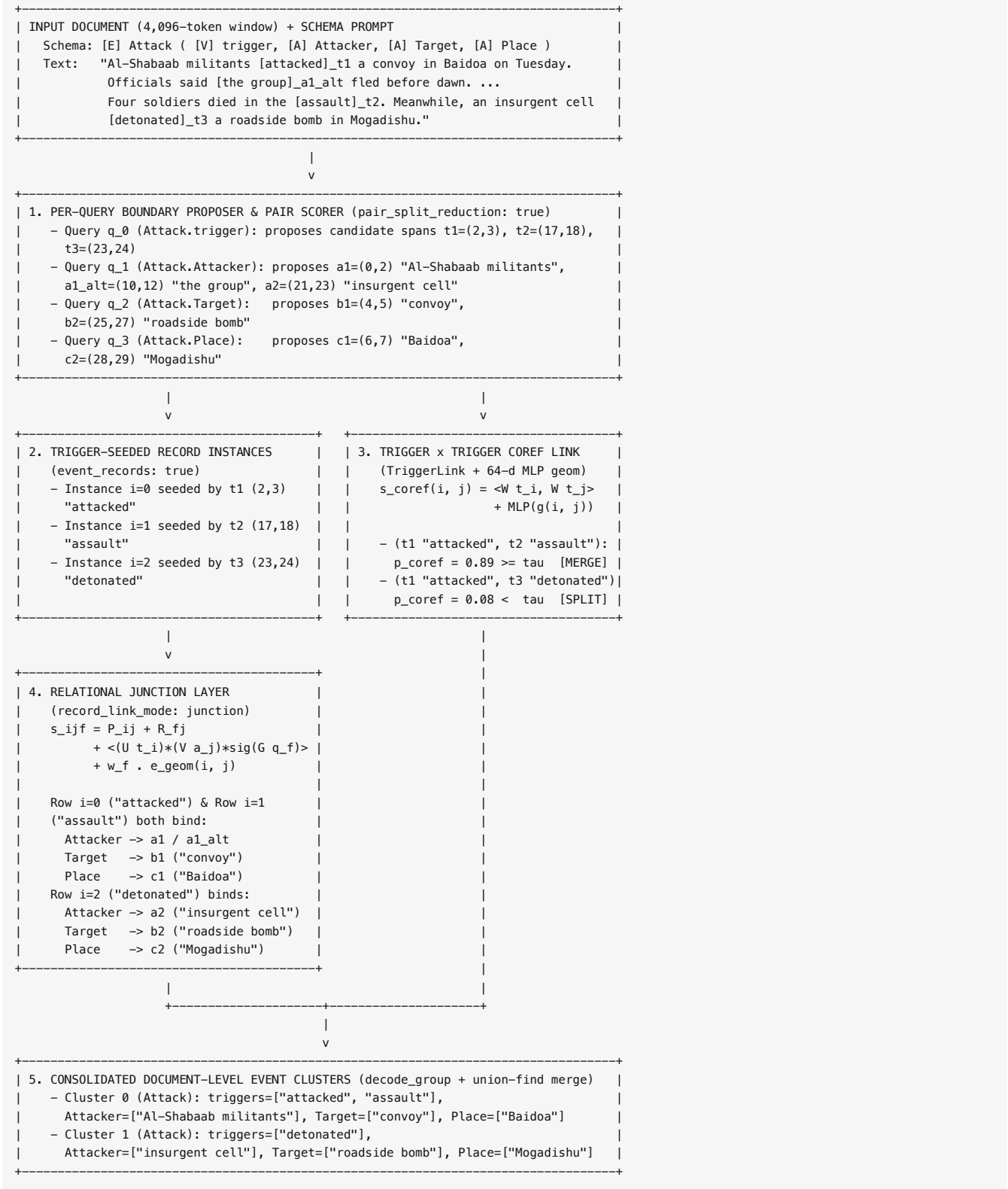
Datasets

- Satyapanich, T., Ferraro, F., Finin, T. (2020). *CASIE: Extracting Cybersecurity Event Information from Text.* AAAI.
- Zhu, M., et al. (2024). *CMNEE: A Large-Scale Document-Level Event Extraction Dataset based on Open-Source Chinese Military News.* LREC-COLING. https://aclanthology.org/2024.lrec-main.299/
- Li, S., Ji, H., Han, J. (2021). *Document-Level Event Argument Extraction by Conditional Generation* (WikiEvents). NAACL. https://aclanthology.org/2021.naacl-main.65/
- Ebner, S., Xia, P., Culkin, R., Rawlins, K., Van Durme, B. (2020). *Multi-Sentence Argument Linking* (RAMS). ACL. https://aclanthology.org/2020.acl-main.718/
- Wang, X., et al. (2020). *MAVEN: A Massive General Domain Event Detection Dataset.* EMNLP. https://aclanthology.org/2020.emnlp-main.129/
- Tong, M., et al. (2022). *DocEE: A Large-Scale and Fine-grained Benchmark for Document-level Event Extraction.* NAACL. https://aclanthology.org/2022.naacl-main.291/
- Li, X., et al. (2020). *DuEE: A Large-Scale Dataset for Chinese Event Extraction in Real-World Scenarios.* NLPCC.
- Zheng, S., Cao, W., Xu, W., Bian, J. (2019). *Doc2EDAG: An End-to-End Document-level Framework for Chinese Financial Event Extraction* (ChFinAnn). EMNLP-IJCNLP. https://aclanthology.org/D19-1032/
- Walker, C., Strassel, S., Medero, J., Maeda, K. (2006). *ACE 2005 Multilingual Training Corpus.* LDC2006T06, Linguistic Data Consortium.
- Tan, Q., Xu, L., Bing, L., Ng, H. T., Aljunied, S. M. (2022). *Revisiting DocRED — Addressing the False Negative Problem in Relation Extraction* (Re-DocRED). EMNLP. https://aclanthology.org/2022.emnlp-main.580/
- Luan, Y., He, L., Ostendorf, M., Hajishirzi, H. (2018). *Multi-Task Identification of Entities, Relations, and Coreference for Scientific Knowledge Graph Construction* (SciERC). EMNLP.
- Luo, L., et al. (2022). *BioRED: A Rich Biomedical Relation Extraction Dataset.* Briefings in Bioinformatics.

Appendix A: Worked Example — Document-Level Event Record with Coreferent Triggers and Arguments

This appendix traces one multi-paragraph document with **two distinct `Attack` events** — the first evoked by two coreferent trigger mentions (`"attacked"` and `"assault"`) and a coreferent argument cluster (`["Al-Shabaab militants", "the group"]`), and the second evoked by a separate trigger (`"detonated"`) in another city — from the raw training JSONL through target compilation, relational junction scoring, `TriggerLink` coreference clustering, and decoded output.

A.1 End-to-End Architecture & Data-Flow Diagram



A.2 Input Training Record JSONL (with Coreferent Triggers and Entity Coreference Clusters)

In the training corpora (`cc_news_events_sonnet55_v2` paired with `cc_news_events_sonnet55_v2_coref_llm` , or offset-gold corpora such as `wikievents_off` and `casie_off`), a document record carries its event annotations under `output.events` (with all coreferent triggers of one event listed in `triggers`) and its entity coreference clusters under `output.coreferences` :


```
{
  "text": "Al-Shabaab militants attacked a convoy in Baidoa on Tuesday. Officials said the group fled before dawn. Four soldiers died in th
  "output": {
    "entities": {
      "Organization": ["Al-Shabaab militants", "insurgent cell"],
      "Facility": ["convoy"],
      "Location": ["Baidoa", "Mogadishu"],
      "Weapon": ["roadside bomb"]
    },
    "coreferences": [
      {
        "canonical": "Al-Shabaab militants",
        "mentions": [
          {"text": "Al-Shabaab militants", "start": 0, "end": 20},
          {"text": "the group", "start": 76, "end": 85}
        ]
      }
    ],
    "events": [
      {
        "event_type": "Attack",
        "trigger": "attacked",
        "triggers": [
          {"text": "attacked", "start": 21, "end": 29},
          {"text": "assault", "start": 130, "end": 137}
        ],
        "arguments": {
          "Attacker": ["Al-Shabaab militants"],
          "Target": ["convoy"],
          "Place": ["Baidoa"]
        }
      },
      {
        "event_type": "Attack",
        "trigger": "detonated",
        "triggers": [
          {"text": "detonated", "start": 168, "end": 177}
        ],
        "arguments": {
          "Attacker": ["insurgent cell"],
          "Target": ["roadside bomb"],
          "Place": ["Mogadishu"]
        }
      }
    ]
  }
}
```

A.3 Compiled Internal Target (`RecordGroupTarget` & `RecordFieldTarget`)

When `expand_argument_coreferents` (`gliner2/processing/argument_coref.py`), `_process_events` (`gliner2/processor.py`), and `_build_record_field_target` (`gliner2/processing/boundary_preprocessing.py`) compile the first `Attack` event above into word-span targets:

1. **Coreferent triggers stay grouped as alternatives of the anchor field (`field_idx = 0`):** `values = ( ((2, 2), (21, 21)), )` — under `record_coreferent_ownership: true` , both `t_1 = (2, 2)` (`"attacked"`) and `t_2 = (21, 21)` (`"assault"`) seed gold instance rows that own the record's arguments, and `(t_1, t_2)` forms a positive pair for `TriggerLink` while `(t_1, t_3)` and `(t_2, t_3)` form hard-negative pairs (`t_3 = (26, 26)` `"detonated"`).
2. **Coreferent argument mentions stay grouped as alternatives of a single filler (`len(ft.values) = 1`):** `expand_argument_coreferents` expands `"Al-Shabaab militants"` with `"the group"` under the same `_coref_group` id, so

`Attacker` compiles to `values = ( ((0, 1), (11, 12)), )` — **one** required filler with **two** acceptable span alternatives (`(0, 1)` `"Al-Shabaab militants"` or `(11, 12)` `"the group"`), rather than two separate required fillers.

```
RecordGroupTarget(
    group_name="Attack",
    mode="natural",
    anchor_field="trigger",
    fields=("trigger", "Attacker", "Target", "Place"),
    instances=(
        # Event 0: coreferent triggers ("attacked" @ (2,2), "assault" @ (21,21))
        RecordInstanceTarget(
            fields=(
                RecordFieldTarget(name="trigger",  values=(((2, 2), (21, 21)),)),
                RecordFieldTarget(name="Attacker", values=(((0, 1), (11, 12)),)),
                RecordFieldTarget(name="Target",   values=(((4, 4),),)),
                RecordFieldTarget(name="Place",    values=(((6, 6),),)),
            )
        ),
        # Event 1: distinct Attack instance ("detonated" @ (26,26))
        RecordInstanceTarget(
            fields=(
                RecordFieldTarget(name="trigger",  values=(((26, 26),),)),
                RecordFieldTarget(name="Attacker", values=(((24, 25),),)),
                RecordFieldTarget(name="Target",   values=(((28, 29),),)),
                RecordFieldTarget(name="Place",    values=(((31, 31),),)),
            )
        ),
    ),
)
```

A.4 Decoded Inference Output JSON (`event_extraction`)

At inference time (`model.extract(text, schema, include_confidence=True, include_spans=True)`), `decode_group` scores each trigger instance, binds arguments through the junction logits `s_ijf` , merges coreferent trigger instances whose `TriggerLink` probability clears the calibrated threshold `τ_coref` into a single cluster (unioning `triggers` and deduplicating role fillers by highest assignment probability), and emits:

```
{
  "event_extraction": {
    "Attack": [
      {
        "trigger": {
          "text": "attacked",
          "start": 21,
          "end": 29,
          "confidence": 0.91
        },
        "triggers": [
          {"text": "attacked", "start": 21, "end": 29, "confidence": 0.91},
          {"text": "assault", "start": 130, "end": 137, "confidence": 0.84}
        ],
        "arguments": {
          "Attacker": [
            {
              "text": "Al-Shabaab militants",
              "start": 0,
              "end": 20,
              "confidence": 0.88,
              "pair_confidence": 0.79
            }
          ],
          "Target": [
            {
              "text": "convoy",
              "start": 32,
              "end": 38,
              "confidence": 0.86,
              "pair_confidence": 0.81
            }
          ],
          "Place": [
            {
              "text": "Baidoa",
              "start": 42,
              "end": 48,
              "confidence": 0.93,
              "pair_confidence": 0.89
            }
          ]
        }
      },
      {
        "trigger": {
          "text": "detonated",
          "start": 168,
          "end": 177,
          "confidence": 0.94
        },
        "triggers": [
          {"text": "detonated", "start": 168, "end": 177, "confidence": 0.94}
        ],
        "arguments": {
          "Attacker": [
            {
              "text": "insurgent cell",
              "start": 153,
              "end": 167,
              "confidence": 0.85,
              "pair_confidence": 0.77
            }
          ],
          "Target": [
            {
              "text": "roadside bomb",
              "start": 180,
              "end": 193,
              "confidence": 0.90,
              "pair_confidence": 0.84
            }
          ],
```

```
          "Place": [
            {
              "text": "Mogadishu",
              "start": 197,
              "end": 206,
              "confidence": 0.95,
              "pair_confidence": 0.92
            }
          ]
        }
      }
    ]
  }
}
```